

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Ramya S S	Reg. No.:	192421226
Section / Batch:	B.Tech IT	Date:	27.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Q30. String Permutation with Constraints

Aim

To generate all permutations of a given string using Backtracking, while ensuring that specified characters do not appear adjacent to each other.

Problem Definition

Given a string and a set of adjacency restrictions, generate all possible permutations such that no restricted pair of characters occurs next to each other.

Example

- Input String: ABC
- Restriction: A and B should not be adjacent.
- Valid Permutations: ACB, BCA
- Invalid Permutations: ABC, BAC

Input

- S – Input string of n characters.
- RestrictedPairs – Set of character pairs that cannot be adjacent.

Output

All valid permutations satisfying the given adjacency constraints.

Backtracking Algorithm

BACKTRACK(path, remaining)

if remaining is empty:

 print path

 return

for each character ch in remaining:

 if path is not empty:

 last = last character of path

if (last, ch) is a restricted pair:

continue // Pruning

add ch to path

remove ch from remaining

BACKTRACK(path, remaining)

remove ch from path

add ch back to remaining

Python Pseudocode

def backtrack(path, remaining):

if not remaining:

print("".join(path))

return

for ch in remaining:

if path and (path[-1], ch) in restricted:

continue

path.append(ch)

remaining.remove(ch)

backtrack(path, remaining)

remaining.append(ch)

path.pop()

S = input("Enter string: ")

restricted = {("A", "B"), ("B", "A")}

backtrack([], list(S))

Manual Process

For S = ABC and restriction A-B:

1. Start with an empty permutation.
2. Choose A.
3. B cannot be selected next because A-B is restricted.
4. Choose C $\rightarrow$ AC.
5. Choose B $\rightarrow$ ACB
6. Backtrack and try another starting character.
7. Choose B, then A is not allowed immediately after B.
8. Choose C $\rightarrow$ BC, then A $\rightarrow$ BCA
9. Invalid branches are stopped immediately using pruning.

How Constraints Reduce Permutation Space

Without restrictions, the number of permutations is:

```
genui {"learning_viz":{"type_id":"PERMUTATION_FORMULA","locale_override":"en-US"}}
```

$n!$

For 3 characters:

$3! = 6$ possible permutations.

With the restriction A-B, only:

ACB, BCA

are valid, so the search space is reduced from 6 to 2 permutations.

Correctness

- The algorithm considers every unused character at each position.
- A permutation is generated only when it satisfies the adjacency restriction.
- Invalid branches are pruned as soon as a restricted adjacent pair is detected.
- Backtracking restores the previous state and explores other possibilities.
- Therefore, all and only valid permutations are generated.

Time and Space Complexity

- Worst-case time: $O(n \times n!)$
- Space: $O(n)$ for the recursion path, excluding the output.

Conclusion

The backtracking approach efficiently generates permutations while avoiding invalid arrangements. Pruning reduces unnecessary exploration, and systematic backtracking ensures completeness and correctness.

Code of Conduct

I Ramya S S/192421226 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ramya S S

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____